

Storage Wars

Deleted isn't gone at Cloudflare, and Redshift's materialized views go open

Issue #5

Fri, 09 Oct 2026

~36 min read

Level 3/5

Today's two stories

1. **A Cloudflare Containers customer could read other customers' leftover disk data, including whole SQLite databases.** One storage option, `skip_block_zeroing`, told the Linux volume manager not to wipe recycled disk blocks before handing them to the next container. Cloudflare fixed it within hours of the report and says it found no sign of abuse. We explain how thin provisioning works, why a 4 KiB write could expose 60 KiB of someone else's data, how the researchers proved the bytes weren't theirs, and why "deleted" data lingers at every layer of the stack, your database included.
2. **Amazon Redshift can now write a materialized view as an open Apache Iceberg table.** With `CREATE MATERIALIZED VIEW ... USING ICEBERG`, Redshift computes a query once, stores the result in S3 as Iceberg, refreshes it incrementally, and lets Athena, Spark, Trino or Snowflake read it. We look at how incremental refresh works on Iceberg snapshots, what forces a full recompute, and the long list of limits in the launch docs.
3. **On the radar:** 6 more things that happened, listed at the end.

STORY 1 Cloudflare's leaky disks: when "deleted" data moves in next door

CLOUDFLARE CONTAINERS & SANDBOXES · DISCLOSED 24 SEP 2026 · LINUX DM-THIN · ~17 MIN READ

Picture renting a flat where the previous tenant's boxes are still stacked in a cupboard you're not supposed to open. That's roughly what a security researcher found on Cloudflare's container platform in September. He started a container, wrote a few kilobytes to its empty disk, read the disk back, and found far more than he'd written: directory listings, database pages and complete SQLite databases that belonged to other Cloudflare customers. No isolation boundary was broken. The data was simply never erased.

WHAT HAPPENED

On **24 September 2026**, Cloudflare published a post titled "How Cloudflare addressed a cross-tenant data exposure vulnerability in Containers".¹ Its `Containers` ≈ Cloudflare's service for running your own Docker-style container images on its network, controlled from Workers code product was affected, and so was **Sandboxes**, which the post says is "built on Containers".¹

The flaw was reported on **4 September** through Cloudflare's `bug bounty` ≈ a programme that pays outside researchers for responsibly reporting security flaws programme on HackerOne by **Oren Yomtov**, a researcher at the

company Accomplish.¹ In plain terms, a customer on a Workers Paid account could recover leftover disk blocks that other customers' containers had used on the same server. Those blocks could hold "filesystem metadata, directory structures, database pages, and application data".¹ BleepingComputer, which covered the disclosure on 27 September, adds that exploitation could have exposed files such as SQLite databases, Chromium browser profiles, `.env` files and credential files.²

Cloudflare says it "has fully remediated the vulnerability, and we have no evidence that customer data has been compromised". It also says customers don't need to do anything.¹ InfoQ covered the story on 5 October.³ The cause was a single option in the storage configuration, `skip_block_zeroing`. To see why one flag could do this, you need to know how Cloudflare gives each container its disk.

THE BACKGROUND YOU NEED

One server, many customers, and a VM around each container

Cloud platforms are **multi-tenant** *≈ serving many customers ("tenants") from the same physical machines, so each must be kept strictly apart from the others*. On Cloudflare Containers, each container runs inside its own **Firecracker** *≈ an open-source virtual machine monitor from AWS that starts very small, fast VMs ("microVMs") using Linux's built-in KVM hypervisor* virtual machine.¹ Firecracker's own README describes microVMs as combining "the security and isolation properties provided by hardware virtualization technology with the speed and flexibility of containers".⁴ A VM boundary is strong. Code inside one VM can't read another VM's memory. But each VM still needs a disk, and those disks are carved out of storage on the host machine. This is where the flaw lived: below the VM, in the code that hands out disk space. InfoQ makes the same point: the bug sat in the storage allocator underneath the Firecracker boundary, not in the boundary itself.³

Thin provisioning: promise lots of disk, hand it out lazily

Containers start and stop all the time, and most never fill their disk. Reserving the full size for each one up front would waste most of the server's storage. So Cloudflare uses **thin provisioning** *≈ giving a virtual disk a large nominal size but only allocating real storage when data is actually written*, through the Linux kernel's **dm-thin** target.¹

Device mapper *≈ the Linux kernel framework that builds virtual block devices on top of real ones; LVM, disk encryption and thin provisioning are all built on it* has a thin-provisioning target whose kernel documentation describes it as a set of targets that "implement thin-provisioning and snapshots".⁵ It works with two pieces:

- A **pool**, which ties together a metadata volume and a data volume. The design "allows many virtual devices to be stored on the same data volume".⁵
- **Thin devices**, the virtual disks carved from that pool. Each one has a map from "virtual block number" to "physical block in the pool". At first the map is empty.

When a thin device writes somewhere it has never written before, the pool grabs a free physical block and records the mapping. That's called **provisioning** a block. When a thin device is deleted, its blocks go back to

the pool for the next device. In Cloudflare's setup, that pool was "shared by multiple customer accounts" on the host.¹

Blocks in a pool aren't the 4 KiB you might expect from a filesystem. The kernel docs say the pool's data block size "must be between 128 (64KiB) and 2097152 (1GiB) and a multiple of 128 (64KiB)".⁵ Cloudflare's affected pools used **64 KiB** blocks, the smallest allowed.¹ Keep that size in mind: it's the reason a small write could expose a much bigger chunk of someone else's data.

Zeroing: the safety step, and the switch that turns it off

What's in a freshly provisioned block? Whatever the last owner left there, unless someone wipes it. By default, dm-thin wipes it. LVM's manual for thin pools says: "When a thin pool provisions a new data block for a thin LV, the new block is first overwritten with zeros."⁶ LVM's generic thin profile sets `thin_pool_zero = 1`.⁷

But zeroing costs time. Every first write into a new 64 KiB block becomes a 64 KiB write. LVM's manual says so plainly: "Provisioning of large zeroed chunks impacts performance."⁶ So dm-thin offers a feature argument that turns it off. The kernel documentation describes `skip_block_zeroing` in one line: "Skip the zeroing of newly-provisioned blocks."⁵ The same document even gives sizing advice for people who do that: "If you are not zeroing newly-allocated data, a larger `$data_block_size` in the region of 262144 (128MiB) is suggested."⁵

Skipping zeroing is safe in exactly one situation: when every byte in the pool belongs to the same owner. A single team running VMs on its own server might reasonably choose it. A shared pool serving many customers is the opposite case.

It isn't only a security problem, either. The Proxmox virtualization project has been adding warnings since February 2026 that its thin pools "must" have zeroing enabled, because without it "certain operations for VM images may lead to data corruption".⁸

Cloudflare's post doesn't say why `skip_block_zeroing` was set on its pools, and doesn't discuss the performance cost of turning zeroing back on.¹

HOW IT WORKS

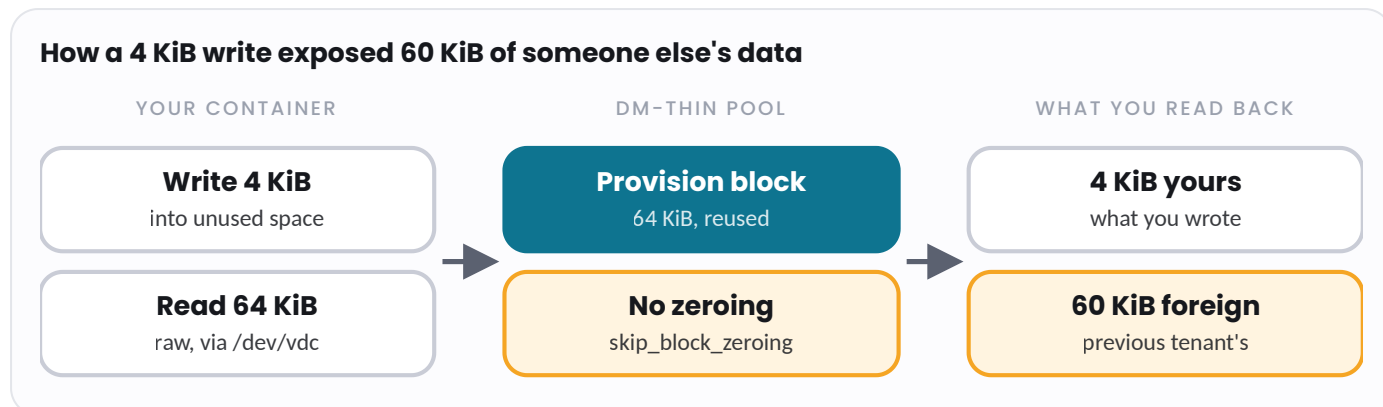
The attack, step by step

Inside a container, the root disk appears to the VM as the block device `/dev/vdc`.¹ A **block device** *≈ a disk as the operating system sees it: an array of fixed-size blocks you can read or write by number, with no files or folders* can be read directly, underneath the filesystem, if you have permission. Inside your own VM, you do.

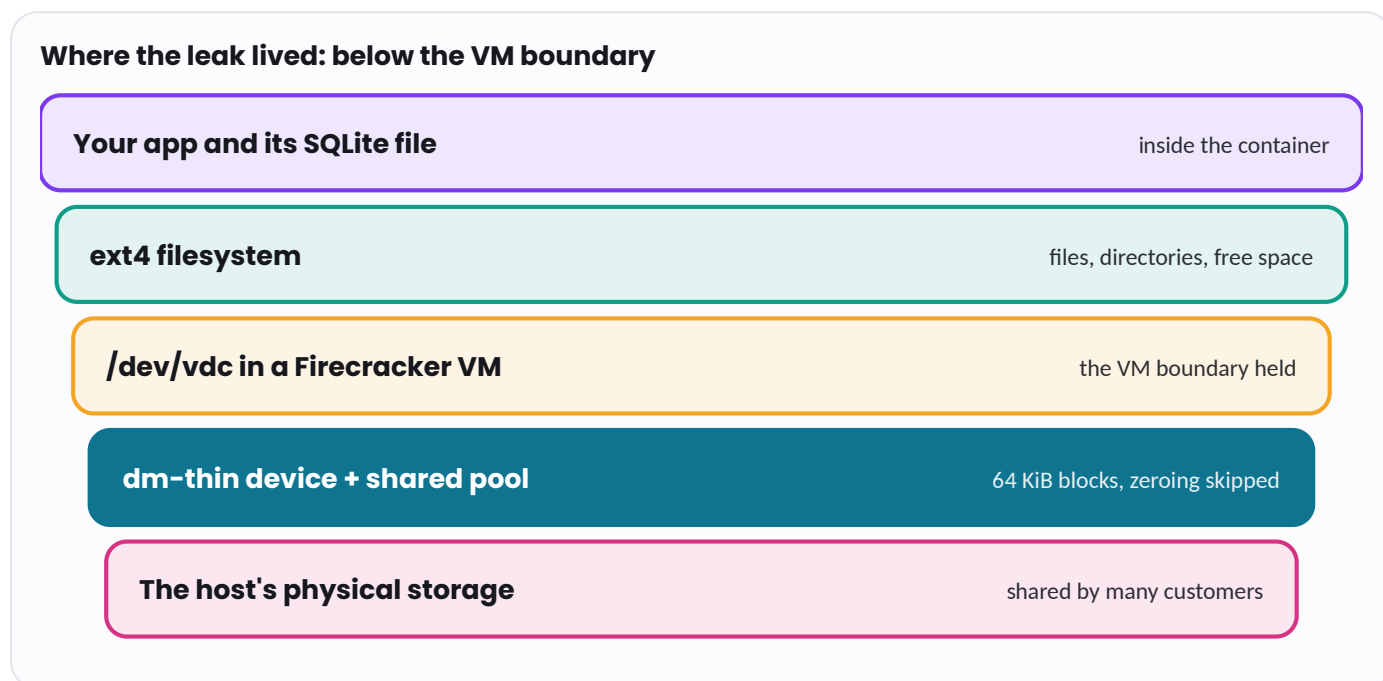
The proof of concept, as Cloudflare describes it, went like this:¹

1. Create a container on a Workers Paid account.
2. Open `/dev/vdc` and read the disk to record a baseline.
3. Write one 4 KiB block into each selected 64 KiB region that sits in the filesystem's free space.
4. Read those regions again.
5. Look only at the parts the container didn't write itself.

Before step 3, reading an unprovisioned region returned zeros, because there was no physical block behind it yet. Step 3 changed that. A 4 KiB write into an unmapped region made dm-thin provision a whole 64 KiB block from the shared pool. Only the 4 KiB that was written got overwritten. With zeroing skipped, the other **60 KiB** still held whatever the block's previous owner had left there, and step 4 read it back.¹



There was a second path, and it made the fix harder. Each host keeps a cache of prepared dm-thin [snapshots](#) *≈ cheap point-in-time copies; in dm-thin a snapshot shares the original's blocks until one side writes* for [OCI image layers](#) *≈ the stacked, read-only filesystem layers that make up a container image, in the standard Open Container Initiative format*, so new containers can start quickly from them. A new container could inherit block mappings from those cached layers without any fresh allocation. Any residual bytes already inside those blocks came along too.¹ So turning zeroing back on, which only affects new allocations, wouldn't clean blocks that were already mapped into running disks or cached layers.



How do you prove the bytes aren't yours?

Here's the clever part. Reading leftover bytes is one thing. Proving to Cloudflare that they came from *other customers*, without actually looking at those customers' data, is another. The researchers used a feature of the ext4 filesystem.

Modern ext4 can store **checksums** *≈ short numbers computed from a block's contents, used to detect corruption* on its metadata, a feature called `metadata_csum`. The kernel's ext4 documentation says the checksum on a directory block is computed over "UUID + inode number + inode generation + the directory block up to the fake entry enclosing the checksum field".⁹ The **UUID** *≈ a unique random ID stamped on each filesystem when it's created* is different for every filesystem, and an **inode** *≈ the record ext4 keeps for each file or directory; its number identifies it within one filesystem* number identifies the directory.

That means a directory block's checksum only verifies against the filesystem that wrote it. The researchers could compute the checksum using their own filesystem's UUID. If it matched, the block was theirs. If the block was clearly a directory block but didn't match, it came from a different filesystem: someone else's. Their scripts reported only aggregate counts and format checks, and the material they submitted contained no recovered content or third-party identifiers.¹

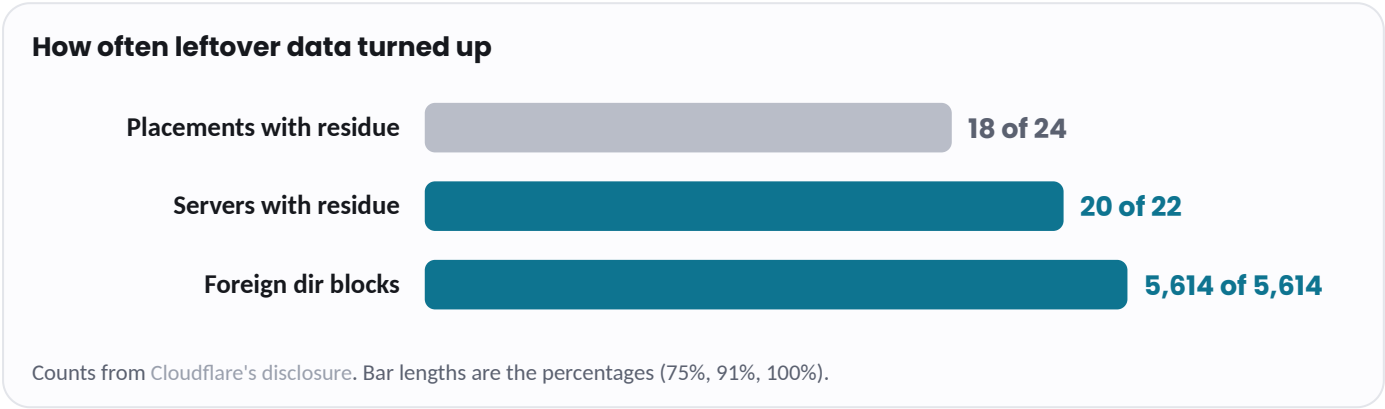
They checked the method first. On blocks they had created and deleted themselves in a test filesystem, "The method correctly attributed all 162 blocks to that filesystem."¹

What they found

The numbers from six production placements were stark:¹

- **All 5,614** testable directory blocks were foreign. **Zero** came from the researchers' own filesystem.
- They identified **2,700** distinct foreign directory inodes.
- Residual material showed up on **18 of 24** placements and **20 of 22** underlying servers, across four continents.

The recovered material included directory structures, database pages and "structurally complete SQLite databases".¹



There were real limits. An attacker "could not select a particular victim or access an actively attached disk".¹ They couldn't choose a customer, workload, host or piece of data, and there was no guarantee that any given block held anything. Which leftovers you got depended on where Cloudflare placed your container and which freed blocks dm-thin handed out. The researchers didn't show any way to modify another customer's live data or affect availability.¹ It was a lottery, but one where most tickets won something.

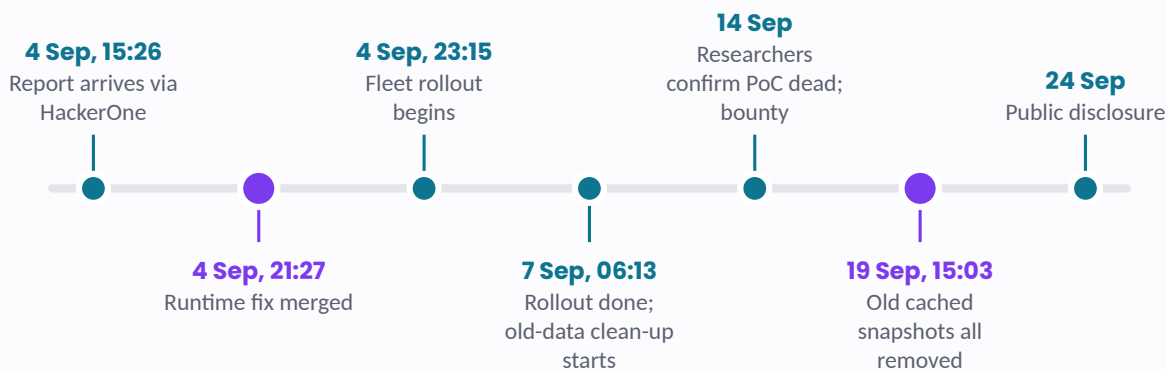
Why a database file is such a good prize

Why would whole SQLite databases turn up in leftover disk blocks? Because databases are designed to keep pages around after you delete things.

SQLite *≈ the embedded SQL database that lives in a single file inside your application, used everywhere from phones to browsers* stores a database as a file of fixed-size pages. When you `DELETE` rows, SQLite doesn't normally scrub the bytes. Its documentation says secure deletion is controlled by the `secure_delete` setting, and "When this option is not used, `secure_delete` defaults to off."¹⁰ With it on, "SQLite overwrites deleted content with zeros", at "a small performance penalty" because of the extra I/O.¹⁰ With it off, deleted values sit in the file until the space is reused.

Now stack the layers. Your app deletes a row, but SQLite keeps the old bytes in the file. Your container deletes the file, but ext4 just marks the blocks as free. The container stops, and Cloudflare's FAQ says that "All disk is ephemeral by default. When a Container instance goes to sleep, the next time it starts, it uses a fresh disk from the container image."¹¹ The old disk's blocks return to the pool, still full of bytes. Every layer did the fast thing and assumed the layer below would clean up. With zeroing skipped, the bottom layer didn't either.

From report to clean fleet



Times in UTC, from Cloudflare's post. The bounty amount wasn't disclosed.

The fix, and the clean-up that took two weeks

The first step was simple. Cloudflare removed `skip_block_zeroing` from the dm-thin pool configuration across the fleet, restoring default zeroing. The fix was merged about six hours after the report arrived.¹

Then came the hard part: the dirty blocks already mapped into existing disks and cached layers. Cloudflare retired all running container disks, removed cached image snapshots created before the fix, drained servers during off-peak hours, restarted the VMs and cleared each server's image cache, so that every disk and layer was rebuilt from zeroed allocations.¹ That work finished on **19 September**.¹

To check whether anyone else had found the hole first, Cloudflare looked at its retained disk I/O **telemetry** *≈ operational measurements a system records about itself, here which reads and writes each disk did*. The giveaway pattern is a small write to an unmapped region followed by reads that return far more than was written. Cloudflare built detection signatures for that and found matches only from the researchers and its own engineers. "We saw no

evidence that this specific attack vector was exploited by anyone else." The post doesn't say how far back that telemetry goes.¹

WHAT IT MEANS FOR YOU

If you run on Cloudflare Containers or Sandboxes, Cloudflare says there's nothing to do.¹ It's still worth asking what your containers wrote to local disk before 7 September. If that included secrets, such as API keys in `.env` files or credentials cached by tools, rotating them is cheap insurance. The window and the impact are unknowable from the outside, so treat it as hygiene, not as evidence of compromise.

If you build a multi-tenant platform, this is the lesson. Anywhere storage is recycled between customers, someone must wipe it. That can be zero-on-allocate, wipe-on-release, or encryption with a separate key per tenant so leftovers are unreadable. One practitioner quoted by InfoQ, Peter Ward of Visa, called it "tenant isolation broken at the storage layer, not an app bug", and suggested asking whether every reused volume path zeroes on allocate.³ Another, Sherin Shahanas, pointed out that "'Isolated' is usually an architectural claim, not something independently tested at the storage layer."³ Cloudflare's own detection idea doubles as a test you can run: allocate, write a little, read a lot, and check that you only see zeros.

For comparison, here's how AWS describes its block storage. The Amazon EBS documentation says the service "ensures that the devices are logically empty (that is, the raw blocks are zeroed or contain cryptographically pseudorandom data) before any customer use or re-use".¹² EBS encryption also uses a unique data key for each volume.¹² That second layer matters. If every tenant's data is encrypted with its own key, a leftover block is just noise to anyone else, even when a wipe is missed.

Your own database leaves data behind too

None of this is unique to clouds. SQLite keeps deleted content in the file unless `secure_delete` is on.¹⁰ Most databases work similarly: `DELETE` marks space as reusable rather than erasing it, which is why it's fast. If you store sensitive data and a disk or file might leave your control (a reused cloud volume, a backup, a laptop), rely on encryption at rest, not on deletion. In SQLite, if you need deleted values to really go, turn the setting on:

```
PRAGMA secure_delete = ON; -- overwrite deleted content with zeros
```

Watch out for

- **This wasn't a VM escape.** The Firecracker boundary held. The leak was in how the host recycled disk space, so "we use microVMs" doesn't by itself mean "tenants are isolated".
- **Turning zeroing on doesn't clean old data.** It only affects new allocations. Blocks already mapped into running disks or cached image layers kept their contents, which is why Cloudflare had to rebuild every disk.¹
- **"No evidence of exploitation" has limits.** It rests on the telemetry Cloudflare kept, and the post doesn't say how far back that goes.¹
- **Fast options exist for single-owner setups.** `skip_block_zeroing` is a legitimate kernel feature.⁵ The danger is using it where blocks move between owners who shouldn't see each other's data.

WHAT HAPPENS NEXT

Cloudflare's post closes with thanks to the researchers and a commitment to act "swiftly and transparently". It doesn't list longer-term changes, such as per-tenant encryption of container disks.¹ Isolation for untrusted code is a hot product category right now. InfoQ notes that the disclosure landed days after Microsoft made Azure Container Apps Sandboxes generally available, and argues that when vendors market isolation for untrusted agent workloads, the guarantee has to extend below the layer named in the announcement.³

Words from this story

Containers (Cloudflare): Cloudflare's service for running your own Docker-style container images on its network, controlled from Workers code.

Bug bounty: a programme that pays outside researchers for responsibly reporting security flaws.

Multi-tenant: serving many customers ("tenants") from the same physical machines, so each must be kept strictly apart from the others.

Firecracker: an open-source virtual machine monitor from AWS that starts very small, fast VMs ("microVMs") using Linux's built-in KVM hypervisor.

Thin provisioning: giving a virtual disk a large nominal size but only allocating real storage when data is actually written.

Device mapper: the Linux kernel framework that builds virtual block devices on top of real ones; LVM, disk encryption and thin provisioning are built on it.

Snapshots (dm-thin): cheap point-in-time copies that share the original's blocks until one side writes.

OCI image layers: the stacked, read-only filesystem layers that make up a container image, in the Open Container Initiative format.

Block device: a disk as the operating system sees it, an array of fixed-size blocks read or written by number.

Checksums: short numbers computed from a block's contents, used to detect corruption.

UUID: a unique random ID stamped on each filesystem when it's created.

Inode: the record ext4 keeps for each file or directory; its number identifies it within one filesystem.

SQLite: the embedded SQL database that lives in a single file inside your application.

Telemetry: operational measurements a system records about itself.

STORY 2 Redshift's materialized views move into the open lake

AMAZON REDSHIFT · ANNOUNCED 5 OCT 2026 · APACHE ICEBERG · ~14 MIN READ

Every data team has a query like this: daily revenue by region, joined from orders and customers, needed by a dashboard, a Spark job, a notebook and a finance analyst's ad-hoc tool. Very often each of those runs the same aggregation from scratch, on its own engine, on its own bill, and they disagree in the third decimal place. Amazon's answer, announced on 5 October, is to compute it once in Redshift and save the result in a format all of them can read.

WHAT HAPPENED

On **5 October 2026**, AWS announced that Amazon Redshift "can now create and refresh Apache Iceberg materialized views".¹³ You write `CREATE MATERIALIZED VIEW ... USING ICEBERG`, and Redshift stores the result as an **Apache Iceberg** *≈ an open table format that turns a folder of data files in object storage into a proper table with snapshots, schema changes and safe concurrent writes* table in Amazon S3 or in S3 table buckets, registered in the **AWS Glue Data Catalog** *≈ AWS's shared catalog that records which tables exist, where their files live and what their schemas are, so many engines can find them*.¹³

Redshift refreshes these views **incrementally** *≈ updating only what changed since last time, instead of recomputing everything*. Any Iceberg-capable engine can read the result. The announcement names Amazon Athena, Apache Spark on Amazon EMR and AWS Glue, Trino and Snowflake.¹³ An AWS Big Data Blog post the same day, "Materialize once, query anywhere", walks through the feature.¹⁴

There's a hardware catch. The feature works on Redshift Serverless and on provisioned clusters with the new **RG** instance types. The older RA3 and DC2 types aren't supported.¹⁵

THE BACKGROUND YOU NEED

Materialized views in one paragraph

A normal **view** *≈ a saved query that behaves like a table; it runs the underlying query every time you read it* stores nothing. A **materialized view** *≈ a view whose result is computed ahead of time and stored, so reading it is fast; it must be refreshed when the source data changes* stores its result. Reads become cheap, but the stored result gets out of date as the source tables change. You have to **refresh** *≈ bring a materialized view's stored result back in line with its source tables* it, and the art is in refreshing cheaply.

Redshift has had materialized views for years. Until now they lived in **Redshift Managed Storage (RMS)** *≈ Redshift's own internal storage layer; fast, but only Redshift can read it*.¹⁵ That's great for Redshift dashboards and useless to a Spark job.

Iceberg in three ideas

In the **data lake** *≈ a large store of files (often Parquet) in cheap object storage like S3, queried by many different engines* world, open table formats such as Iceberg are how a pile of files becomes a table. Three ideas matter here:

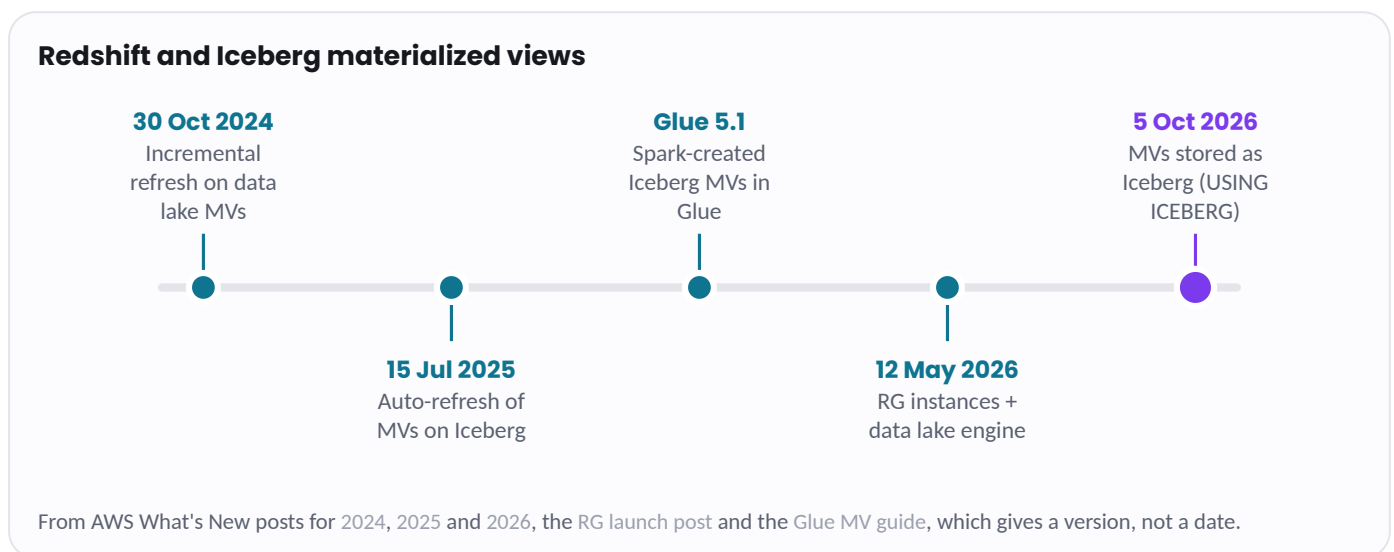
1. **Data lives in ordinary files.** Rows are stored in **Parquet** *≈ a column-oriented file format: values from each column are stored together, so queries read only the columns they need* files in object storage.
2. **Every change creates a snapshot.** An Iceberg table is a series of snapshots. Each write adds a new one that lists exactly which data files make up the table at that moment. Old snapshots stick around until they're **expired** *≈ deleted on purpose by a maintenance job, along with any files only they referenced*.
3. **A catalog says which snapshot is current.** Engines ask the catalog (here Glue) for the table's current metadata, then read the files it points to.

The second idea is the one that makes incremental refresh possible. If you remember which snapshot of each source table you last read, you can ask Iceberg which files were added or removed since then.

How we got here

AWS has been building towards this for a while. Redshift gained incremental refresh for materialized views on data lake tables, Iceberg included, on 30 October 2024. Before that, "changes to open table format data ... required a full recompute".¹⁶ On 15 July 2025 it added automatic refresh of materialized views on Iceberg tables.¹⁷ Those views still stored their results inside Redshift.

Meanwhile AWS Glue 5.1 gained its own Iceberg materialized views, created from Spark, stored as Iceberg tables in S3 and readable by Athena and Redshift.¹⁸ In May 2026 Redshift launched the Graviton-based RG instance family with a built-in **data lake query engine** *≈ the part of Redshift that reads Iceberg and Parquet files in S3 directly, instead of only its own internal storage*, which runs lake queries on the cluster's own nodes. AWS claims performance "up to 2.4x as fast as RA3 for Apache Iceberg", says RG runs warehouse workloads faster "at 30% lower price per vCPU", and says data lake queries "incur zero per-terabyte scanning charges". Those are AWS's own numbers.¹⁹



HOW IT WORKS

Creating one

The Big Data Blog's main example builds a daily revenue view. Here is a version of it, assembled from the fragments in the post. The exact column list is illustrative:¹⁴

```
CREATE MATERIALIZED VIEW awsdatalog.analytics.daily_revenue
USING ICEBERG
LOCATION 's3://amzn-s3-demo-analytics/daily_revenue/'
PARTITIONED BY (day(order_date))
AS
SELECT region, order_date,
       SUM(amount) AS total_revenue,
       COUNT(*)    AS transaction_count
FROM awsdatalog.sales.orders
GROUP BY region, order_date;
```

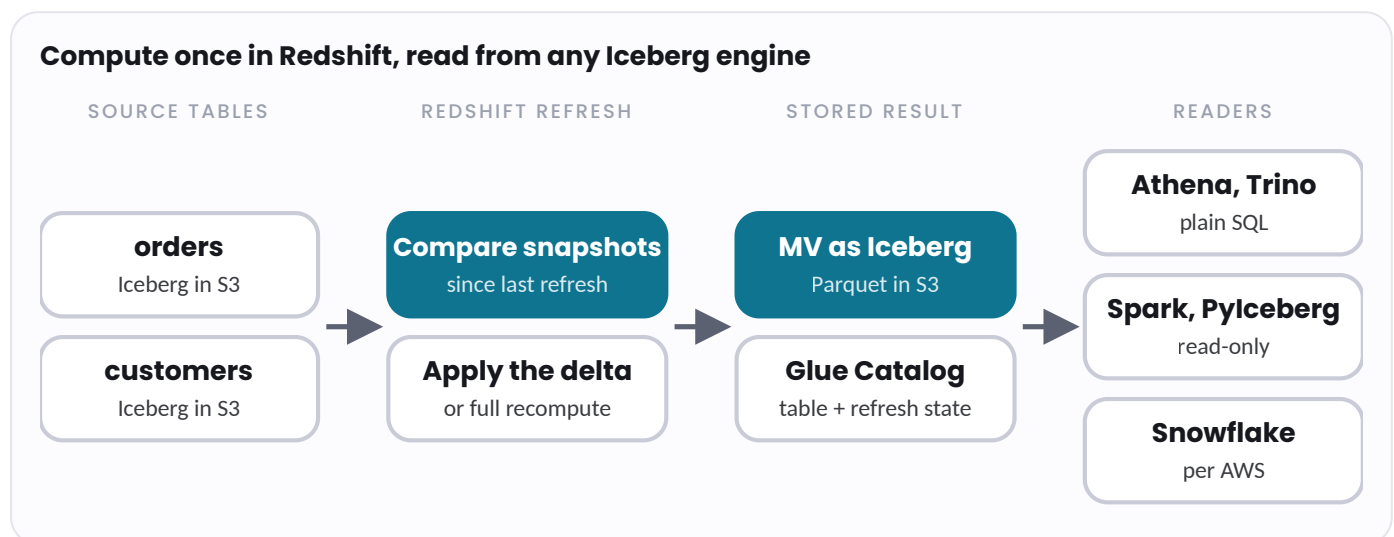
The documentation says Redshift then does three things:¹⁵

1. It runs the defining query and writes the results as Parquet data files in S3.
2. It registers the table in the Glue Data Catalog "with Iceberg metadata, making it discoverable by other engines".
3. It stores the view's definition and refresh state in Glue "for cluster-independent management".

If you leave out `LOCATION` and use S3 Tables, S3 Tables manages the storage for you.¹⁴ The source tables must be Iceberg tables in the same AWS Region and account.¹⁵

Reading it from anywhere

To everyone except Redshift, the result is just an Iceberg table. The docs say other engines "can read them using standard Iceberg table access patterns without any additional configuration".¹⁵ The blog shows the same table read from Athena with plain SQL, from Spark with `spark.table("analytics.daily_revenue")`, and from Python with [Pylceberg](#) *≈ the official Python library for reading and writing Iceberg tables without a big engine*. The blog says readers pay standard S3 read costs.¹⁴



Refresh: comparing snapshots

When you run `REFRESH MATERIALIZED VIEW`, Redshift "compares the current source table Iceberg snapshots against the snapshots recorded at the last refresh". If something changed, it applies only the changed data (incremental refresh) or recomputes the whole thing (full refresh).¹⁵

Why only some queries qualify for the incremental path comes down to arithmetic. Take `SUM(amount) GROUP BY region`. If new orders arrive, you add their amounts to the right regions. If orders are deleted, you subtract them. `COUNT` works the same way. You never need to look at the rest of the table. Now try `MAX(amount)`. A new order is easy, since you compare it with the current max. But if the order that *was* the max gets deleted, the only way to find the new max is to scan every remaining row in that group. `DISTINCT`, window functions and outer joins have the same problem in different forms: one changed row can affect results far away from it.

The documentation's lists follow that logic. Incremental refresh is supported for:¹⁵

- `SELECT ... FROM ... WHERE ... GROUP BY` with `COUNT` and `SUM`
- Inner joins between Iceberg source tables
- `UNION ALL`

These constructs are allowed, but force a full refresh: `DISTINCT`, outer joins, window functions, subqueries, `INTERSECT` and `EXCEPT`, `GROUPING SETS` / `ROLLUP` / `CUBE`, aggregates other than `COUNT` and `SUM` (so `MIN`, `MAX` and `AVG`), and `COUNT(DISTINCT)` / `SUM(DISTINCT)`.¹⁵

Redshift also falls back to a full refresh when the source snapshots it recorded last time have expired, or when someone modified the view's data outside a Redshift refresh.¹⁵ The first case is the one to plan for. If a nightly maintenance job expires source snapshots older than a day and you refresh weekly, every refresh will be a full one. The docs advise configuring "snapshot retention to exceed your expected refresh interval".¹⁵



The comparison above comes from the docs and the blog. The blog suggests using both together: build pipelines as Iceberg materialized views, then load the hottest results into classic RMS views for low-latency dashboards.¹⁴

Many clusters, one view

Because the refresh state lives in Glue rather than inside a cluster, any Redshift cluster or Serverless workgroup with the right permissions can refresh the view. What if two try at once? Redshift uses optimistic concurrency control *≈ letting writers proceed without locks, then checking at commit time whether someone else got there first;*

if so, one of them backs off through Glue's conditional update mechanism, so only one refresh succeeds. The other then checks whether the view is still stale. If it's already fresh, it returns success without redoing the work.¹⁵

Small files and housekeeping

Incremental refresh writes small batches of new files each time. Over weeks, a table made of thousands of tiny Parquet files reads slowly. That's why Iceberg tables need **compaction** *≈ rewriting many small data files into fewer large ones*. If the view lives in S3 table buckets, S3 Tables compacts it automatically. If it lives in a general-purpose bucket, Redshift doesn't compact it, and the docs tell you to run something like Spark's `rewrite_data_files` procedure regularly and configure snapshot expiration. Redshift recognises compaction as maintenance and doesn't treat it as a change that forces a full refresh.¹⁵

The privacy wrinkle nobody expects

The docs include a section you rarely see in a launch: snapshot history visibility. Because the view is a normal Iceberg table with full snapshot history, "A reader with access to the underlying Iceberg files can compare successive snapshots to observe which rows were added or removed between refreshes."¹⁵ For an aggregated view, that only reveals changes in totals. For a non-aggregated view, it reveals exactly which rows changed. The suggested mitigations are to lock down the files, expire snapshots aggressively, or deliberately use a construct like `DISTINCT` that forces a full refresh, so each snapshot replaces all the data and the differences say nothing.¹⁵

WHAT IT MEANS FOR YOU

If you're on AWS and several engines compute the same aggregates, this is the use case AWS built it for. The blog's business case compares about \$7,500 a month for four engines each recomputing the same 1 TB aggregation against about \$1,500 for a single refresh. Those are AWS's illustrative assumptions, and the post says savings vary by workload.¹⁴ The bigger win may be consistency: one definition of "daily revenue", with every tool reading the same numbers.

Check your cluster type first. If you run provisioned RA3 nodes, you can't use this until you move to RG or Serverless.¹⁵

Design the view for the incremental path. Stick to `SUM`, `COUNT`, inner joins and filters where you can. If you need an average, store `SUM` and `COUNT` and divide when you read. Keep source snapshots around longer than your refresh interval, and put the view in S3 table buckets unless you want to run compaction yourself.¹⁵

Designing for incremental refresh

Forces a full refresh

```
SELECT region,
       AVG(amount) AS avg_amt,
       MAX(amount) AS max_amt
FROM sales.orders o
LEFT JOIN sales.customers c
      ON o.cust_id = c.id
GROUP BY region;
```

Stays incremental

```
SELECT region,
       SUM(amount) AS sum_amt,
       COUNT(*)    AS n
FROM sales.orders o
JOIN sales.customers c
      ON o.cust_id = c.id
GROUP BY region;
-- avg = sum_amt / n at read time
```

Illustrative queries. The incremental and full-refresh rules are from the Redshift developer guide. The left one drops back to full refresh on AVG, MAX and the outer join.

Readers can't write back. Only Redshift can refresh or drop a view it created. Iceberg materialized views created by other engines, such as Spark via Glue, can be queried by Redshift as read-only tables, but Redshift can't refresh or drop them.¹⁵ Pick one engine to own each view.

🌟 Watch out for

- **No Iceberg v3 sources.** The limitations list says you can't build these views on "Source tables using Iceberg format version 3".¹⁵
- **The blog and the docs disagree on `UNION ALL`.** The docs list it as incremental. The blog lists it among constructs that force a full refresh.^{15,14} Check `SVL_MV_REFRESH_STATUS` after a refresh to see what actually happened.
- **More limits:** no UDFs, no functions like `GETDATE()` or `RANDOM()`, no `ORDER BY / LIMIT`, no uppercase identifiers, no tables protected by Lake Formation row or column filters, no automatic query rewrite, no `CASCADE` refresh, and it won't work with `enable_case_sensitive_identifier` set to true.¹⁵
- **Refresh is manual today.** The blog's examples all call `REFRESH MATERIALIZED VIEW` by hand.¹⁴ One independent write-up quotes the developer guide as saying "Autorefresh is not supported for Iceberg materialized views".²¹ Schedule the refresh yourself.
- **Dropping the view leaves the files.** `DROP MATERIALIZED VIEW` removes the Glue entry but not the S3 data.¹⁴
- **Refresh history is per cluster.** If several clusters refresh the same view, each one's `SVL_MV_REFRESH_STATUS` shows only its own refreshes.¹⁵

WHAT HAPPENS NEXT

The bigger question is whether "materialized view" becomes part of Iceberg itself. Today Redshift and Glue each keep their own refresh state in Glue. In Apache Iceberg, a **Materialized View Spec** has been under discussion as pull request #11041. It proposes a view that points to a "storage table", with each snapshot of that table recording a refresh state: the view version, when the refresh started, and the snapshot ID of every source table.²⁰ That's very close to what Redshift does. As of early October the PR is still being debated, including the question of how to define a "fresh" view.²⁰ An independent review of AWS's sources, published on 7 October, notes that AWS itself says Iceberg materialized views "are not part of the open-source Apache Iceberg specification".²¹

Until a shared spec exists, "query anywhere" means *read* anywhere. Each view still has exactly one engine that knows how to keep it fresh.

Words from this story

Apache Iceberg: an open table format that turns a folder of data files in object storage into a proper table with snapshots, schema changes and safe concurrent writes.

AWS Glue Data Catalog: AWS's shared catalog that records which tables exist, where their files live and what their schemas are.

Incremental (refresh): updating only what changed since last time, instead of recomputing everything.

View: a saved query that behaves like a table and runs every time you read it.

Materialized view: a view whose result is computed ahead of time and stored, so reading it is fast.

Refresh: bringing a materialized view's stored result back in line with its source tables.

Redshift Managed Storage (RMS): Redshift's own internal storage layer; fast, but only Redshift can read it.

Data lake: a large store of files in cheap object storage like S3, queried by many engines.

Parquet: a column-oriented file format that lets queries read only the columns they need.

Expired (snapshots): deleted on purpose by a maintenance job, along with files only they referenced.

Data lake query engine (RG): the part of Redshift that reads Iceberg and Parquet files in S3 directly.

Pylceberg: the official Python library for reading and writing Iceberg tables.

Optimistic concurrency control: letting writers proceed without locks and checking at commit time whether someone else got there first.

Compaction: rewriting many small data files into fewer large ones.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **PostgreSQL 19 RC still pending:** as of 9 October, the newest PostgreSQL 19 announcement on postgresql.org is still Beta 4 (24 Sep), and the roadmap still says the release "is planned for October 2026".^{22,23}
- **pg_plan_filter 1.0.0 (7 Oct):** a Postgres module for 14–18 that rejects statements, or whole transactions, whose estimated planner cost is above a limit you set.²⁴
- **DynamoDB filtered export:** exports to S3, full or incremental, can now be narrowed with key-condition, filter and projection expressions, so you export one tenant or a few attributes instead of a whole table.^{25,26}
- **Aurora DSQL partial indexes:** `CREATE INDEX . . . WHERE` now works in DSQL (release notes dated 15 Sep), alongside batched nested-loop joins that cut round trips between compute and storage.²⁷
- **ClickHouse Cloud (5 Oct):** Kinesis ClickPipes now accept Avro and Protobuf as well as JSON, and Kafka ClickPipes accept Protobuf schemas as `.proto` or `.desc` files.²⁸
- **Databricks Discover and domains GA (8 Oct):** the Discover page and domains, for finding and organising data assets, are now generally available.²⁹

1. [How Cloudflare addressed a cross-tenant data exposure vulnerability in Containers](#), Rushil Mehra, Cody Roseborough, Avishek Sarkar and Hrushikesh Deshpande, Cloudflare, 24 Sep 2026.
2. [Cloudflare fixes Containers cross-tenant flaw exposing customer data](#), Bill Toulas, BleepingComputer, 27 Sep 2026.
3. [Cloudflare Fixes Cross-Tenant Data Exposure in Containers](#), InfoQ, 5 Oct 2026.
4. [Firecracker README](#), firecracker-microvm project, GitHub.
5. [Thin provisioning](#), Linux kernel documentation (7.1).
6. [lvmthin\(7\)](#), LVM manual page (Fedora 39).
7. [conf/thin-generic.profile](#), LVM2 source (mirror).
8. [close #7204: document and check that thin pools have zeroing enabled](#), Fiona Ebner, pve-devel mailing list, Feb–Sep 2026.
9. [ext4 Checksums](#), Linux kernel documentation.
10. [SQLite documentation source: PRAGMA secure_delete and SQLITE_SECURE_DELETE](#), sqlite.org.
11. [Containers FAQ](#), Cloudflare documentation, viewed 9 Oct 2026.
12. [Data protection in Amazon EBS](#), AWS documentation, viewed 9 Oct 2026.
13. [Amazon Redshift adds support for creating and refreshing Apache Iceberg materialized views](#), AWS What's New, 5 Oct 2026.
14. [Materialize once, query anywhere: Introducing Iceberg materialized views in Amazon Redshift](#), Sudipta Bagchi, Andre Hernich, Gaurav Saxena, Dhaval Shah and Srishti Mittal, AWS Big Data Blog, 5 Oct 2026.
15. [Materialized views stored as Apache Iceberg tables](#), Amazon Redshift Database Developer Guide, viewed 9 Oct 2026.
16. [Amazon Redshift now supports incremental refresh on Materialized Views \(MVs\) for data lake tables](#), AWS What's New, 30 Oct 2024.
17. [Amazon Redshift announces support for automatic refresh of materialized views on Apache Iceberg tables](#), AWS What's New, 15 Jul 2025.
18. [Using materialized views with AWS Glue](#), AWS Glue Developer Guide, viewed 9 Oct 2026.
19. [Amazon Redshift introduces AWS Graviton-based RG instances with an integrated data lake query engine](#), AWS News Blog, 12 May 2026.
20. [Materialized View Spec, apache/iceberg PR #11041](#), Jan Kaul and reviewers, GitHub, viewed 9 Oct 2026.
21. [Apache Iceberg Materialized Views on AWS](#), Hidekazu Konishi, 7 Oct 2026.
22. [PostgreSQL News Archive](#), postgresql.org, viewed 9 Oct 2026.
23. [Roadmap](#), postgresql.org, viewed 9 Oct 2026.

24. [pg_plan_filter 1.0.0 released](#), PGX, postgresql.org, 7 Oct 2026.
25. [Filtered export to Amazon S3](#), Amazon DynamoDB Developer Guide.
26. [DynamoDB S3 Export Now Supports Filter Specification](#), Classmethod DevelopersIO.
27. [Release notes for Aurora DSQL](#), AWS documentation, viewed 9 Oct 2026.
28. [ClickHouse Cloud changelog 2026](#), ClickHouse, 5 Oct 2026 entry.
29. [Databricks platform release notes, October 2026](#), Databricks, 8 Oct 2026 entry.